

Lösungsblatt: CQRS und Event Sourcing

Modul 321 – Lektion 03

Hinweis: Dieses Lösungsblatt enthält Musterlösungen und einen möglichen Erwartungshorizont. Andere Antworten sind korrekt, wenn sie fachlich sauber begründet sind und CQRS sowie Event Sourcing nachvollziehbar anwenden.

1. Begriffe sauber erklären

Begriff	Mögliche Lösung
CQRS	CQRS trennt Schreibzugriffe und Lesezugriffe bewusst in unterschiedliche Verantwortlichkeiten oder Modelle. Dadurch können die Schreibseite auf Fachregeln und Konsistenz und die Leseseite auf schnelle, passende Abfragen optimiert werden.
Command	Ein Command ist die Aufforderung, eine fachliche Änderung auszuführen. Er beschreibt eine Absicht wie zum Beispiel <code>PlaceOrder</code> und kann angenommen oder abgelehnt werden.
Query	Eine Query liest Informationen aus dem System, ohne den fachlichen Zustand zu verändern. Sie dient zum Beispiel der Anzeige, Suche oder Auswertung.
Event Sourcing	Beim Event Sourcing werden nicht nur aktuelle Zustände gespeichert, sondern die fachlichen Ereignisse, die zu diesen Zuständen geführt haben. Der aktuelle Zustand kann aus der Reihenfolge dieser Events wieder aufgebaut werden.

2. Command oder Query?

- `CreateBooking`: Command, weil eine neue Buchung erzeugt und damit der Zustand verändert werden soll.
- `GetOpenBookings`: Query, weil offene Buchungen nur gelesen werden.
- `CancelInvoice`: Command, weil der Status einer Rechnung geändert wird.
- `GetCourseOverview`: Query, weil eine Übersicht für die Anzeige abgefragt wird.

3. Warum CQRS?

1. Ein gemeinsames CRUD-Modell kann unpraktisch werden, weil sehr viele Lesezugriffe und wenige, aber fachlich kritische Schreibzugriffe dieselben Strukturen und dieselbe Datenhaltung belasten.

2. Ein getrenntes Write-Model hilft, weil dort Regeln wie freie Plätze, Fristen oder Doppelanmeldungen sauber geprüft werden können.
3. Ein getrenntes Read-Model hilft, weil Übersichten, Statistiken und Suchansichten gezielt für schnelle Abfragen aufbereitet werden können.
4. Das Read-Model darf anders strukturiert sein, weil es nicht primär Fachregeln durchsetzen muss, sondern eine geeignete Form für UI, Reports oder Suche liefern soll. Daher darf es Daten zusammenfassen, duplizieren oder denormalisieren.

4. Event Sourcing nachvollziehen

1. Aktueller Kontostand: 135. Rechnung: $0 + 150 - 40 + 25 = 135$.
2. Beim Replay wird mit einem Anfangszustand gestartet und danach werden alle Events in der richtigen Reihenfolge erneut angewendet. So entsteht derselbe aktuelle Zustand wieder.
3. Der Vorteil ist die vollständige Nachvollziehbarkeit: Man sieht, wie der Zustand entstanden ist, und kann Projektionen oder Ansichten später neu aufbauen.
4. Ein Nachteil ist die höhere Komplexität, zum Beispiel bei Event-Modellierung, Versionierung oder beim Umgang mit langen Event-Streams.

5. Command oder Event?

1. `PlaceOrder`: Command, weil es die Absicht beschreibt, eine Bestellung zu platzieren.
2. `OrderPlaced`: Event, weil es die eingetretene Tatsache beschreibt, dass die Bestellung platziert wurde.
3. `ReserveSeat`: Command, weil das System aufgefordert wird, einen Platz zu reservieren.
4. `SeatReserved`: Event, weil die Reservation bereits erfolgt ist.

Zusatzfrage: Ein Command kann abgelehnt werden, wenn Regeln verletzt sind, zum Beispiel wenn kein Platz frei ist. Ein Event beschreibt dagegen etwas, das bereits passiert und erfolgreich festgehalten worden ist.

6. CQRS und Event Sourcing kritisch beurteilen

1. Falsch. CQRS bedeutet zuerst eine Trennung von Verantwortungen, nicht zwingend zwei getrennte Anwendungen oder Deployments.
2. Richtig. Read-Modelle oder Projektionen können aus den gespeicherten Events jederzeit neu aufgebaut werden.
3. Falsch. Für einfache CRUD-Formulare ist Event Sourcing oft unnötig kompliziert und bringt wenig Mehrwert.
4. Falsch. Commands beschreiben eine Absicht, Events eine eingetretene fachliche Tatsache.

7. Read-Model entwerfen

1. Ein mögliches Read-Model ist eine Tabelle oder Sicht pro Veranstaltung mit Feldern wie Titel, freie_Plaetze, reservierte_Tickets und bezahlte_Tickets.
2. Mögliche Events: `TicketReserved`, `TicketPaid`, `TicketCancelled`, eventuell `EventCreated` oder `CapacityChanged`.
3. Dieses Read-Model ist für die Anzeige nützlicher, weil die UI die fertigen Kennzahlen direkt laden kann, statt rohe Event-Folgen selbst auszuwerten.

8. Praxisaufgabe

Mögliche Lösungsskizze:

1. Die Lösung kann aus einer Command-Seite für Buchungen und Stornierungen, einem Event Store und mehreren Read-Modellen für Verfügbarkeiten und Auslastungs-Dashboards bestehen.
2. Mögliche Commands: `CreateRoomReservation`, `CancelRoomReservation`, `ConfirmRoomReservation`, `ChangeReservationTime`.
3. Mögliche Events: `RoomReservationCreated`, `RoomReservationCancelled`, `RoomReservationConfirmed`, `ReservationTimeChanged`.
4. Mögliches Write-Model: ein Aggregat pro Reservation oder pro Raum/Zeitslot. Wichtige Regeln: keine Doppelbelegung desselben Raums im selben Zeitfenster, nur gültige Zeiträume, Stornierung nur für bestehende Reservationen, Änderungen nur innerhalb erlaubter Fristen.
5. Mögliche Read-Modelle: erstens eine Verfügbarkeitsansicht pro Raum und Datum für die Benutzeroberfläche; zweitens ein Dashboard mit Auslastung pro Tag oder Woche für die Verwaltung.
6. Event Sourcing unterstützt die Historie, weil jede fachliche Änderung als Event erhalten bleibt. Neue Ansichten können durch Replay oder Projektionen aus denselben Ereignissen aufgebaut werden.
7. Mögliche zusätzliche Herausforderungen: Event-Versionierung, eventual consistency zwischen Write- und Read-Seite, komplexeres Debugging, Umgang mit Replay und langen Event-Streams.

9. Umsetzungsnahe Zusatzaufgabe

Ein möglicher Ablauf: Erst sendet das Frontend den Command `CreateRoomReservation`. Danach validiert die Write-Seite, ob der Raum im gewünschten Zeitraum frei ist. Wenn die Regeln erfüllt sind, wird das Event `RoomReservationCreated` gespeichert. Anschliessend verarbeitet eine Projection dieses Event und aktualisiert das Read-Model für freie Räume. Das Dashboard oder die Raumübersicht liest dann die neue Sicht über eine Query aus. Kurzzeitig kann eventual consistency auftreten, falls die Read-Seite das Event noch nicht verarbeitet hat. Bei einer späteren Stornierung läuft derselbe Mechanismus mit einem neuen Command und einem neuen Event erneut ab.

Didaktischer Hinweis

Wichtig ist nicht, dass Lernende eine einzige Zielarchitektur auswendig reproduzieren, sondern dass sie die Denkrichtung verstehen: CQRS trennt Absichten und Abfragen, Event Sourcing speichert fachliche Geschichte, und beide Muster lohnen sich nur dann, wenn sie ein reales Problem besser lösen als ein einfaches CRUD-Modell.